Lean / Kanban:

Filosofía Lean: una filosofía que busca la mejora contínua de los procesos, principalmente mediante la eliminación de desperdicios.

Principios:

- **Eliminar Desperdicios:** el desperdicio es cualquier actividad que no genere valor. Es el principio central de Lean. Surge con el principio de JIT y se relaciona con los principios ágiles de software funcionando y simplicidad.
- **Amplificar Aprendizaje:** mejora contínua y conocimiento explícito del equipo en base a la experiencia. Se relaciona con el principio ágil de autoorganización.
- **Embeber la Integridad Conceptual:** considerar todas las partes del producto para construir con calidad desde el principio. Se relaciona con el principio ágil de atención contínua a la excelencia.
- **Diferir Compromisos:** tomar decisiones en el último momento posible en el que es responsable hacerlo. Al tomar decisiones tempranas hay incertidumbre.
- **Dar Poder al Equipo:** dar poder a la gente, delegación de decisiones y posibilidades para que puedan hacer sus tareas. Equipos autogestionados.
- **Ver el Todo:** tener claro el producto completo, su arquitectura y el aporte de cada uno.
- **Entregar lo Antes Posible:** entregas rápidas e incrementales para aumentar el feedback y reducir desperdicios.

Categorías de Desperdicios (7 + 1):

Producción Tradicional	**Desarrollo de Software**
Producción en exceso	Características extra
Stock innecesario	Trabajo parcialmente terminado
Pasos extra	Proceso extra (ej: calidad más alta de la requerida)
Búsqueda de información / movimiento	Cambio de contexto (ej: varios proyectos a la vez)
Defectos	Defectos
Esperas	Demoras
Transportes	Transferencia de conocimiento / cambio de tareas
Talento no utilizado	Talento no utilizado (ej: buen desarrollador como PM)

Tipos de Desperdicio:

- **Necesarios (tipo 1):** desperdicios que no aportan valor pero son necesarios para el funcionamiento, como supervisores. Hay que minimizarlos.
- **Puro (tipo 2):** no generan valor ni son necesarios. Hay que eliminarlos.

Kanban: es un método o framework para definir, gestionar y mejorar procesos que entregan trabajo del conocimiento. Su objetivo es mejorar de forma iterativa y contínua los procesos y gestionar los cambios. No es un proceso ni una metodología y no gestiona proyectos ni productos.
Se basa en el uso de un tablero que tenga columnas representando las fases, etapas o estado de las tareas, e ítems en su respectiva columna con un responsable asociado.

Tiene 9 valores, 6 principios y 6 prácticas concretas.

Valores:
- Transparencia
- Equilibrio
- Colaboración
- Foco en el cliente
- Flujo
- Liderazgo
- Entendimiento
- Acuerdo
- Respeto

Diferencia principal con SCRUM: el flujo. No se maneja con proyectos con inicio y fin, es un flujo contínuo de mejora.

Principios:
- **Gestión de Cambios:**
    - Comenzar con lo que hacés ahora: no es disruptivo ni te hace empezar de cero. Parte de los procesos como son en la actualidad respetando los roles y responsabilidades de cada persona para mejorarlos. Reduce resistencia y excusas a su adopción.
    - Acordar la búsqueda de la mejora a través del cambio evolutivo.
    - Fomentar actos de liderazgo a todos los niveles.
- **Entrega de Servicios:**
    - Comprender y enfocarse en cumplir las necesidades y expectativas del cliente.
    - Gestionar el trabajo; dejar que los trabajadores se auto organicen en torno a él.
    - Revisar periódicamente la red de servicios y sus políticas para mejorar los resultados entregados.

Prácticas:
- **Visualizar:** mostrar el trabajo, su flujo, riesgos, como se trabaja, etc.
- **Limitar el Trabajo en Curso:** limita el trabajo en el sistema a la capacidad disponible basándote en datos. Terminar de empezar, empezar a terminar.

- **Gestionar el Flujo:** identificar y resolver los cuellos de botella para tener un flujo de trabajo contínuo.
- **Hacer las Políticas Explícitas:** tener políticas acordadas y visibles para todos los miembros abarcando criterio del pull, WIP, clases de servicio, etc.
- **Establecer Ciclos de Retroalimentación:** retroalimentación con la cadencia adecuada para fomentar el aprendizaje, colaboración y mejoras.
- **Mejorar Colaborativamente, Evolucionar Experimentalmente:** ejecutar experimentos para avanzar o aprender. Permitir que los equipos se organicen de acuerdo a sus conocimientos y habilidades.

Visualizar:
- Permite absorber y procesar una gran cantidad de información en un corto intervalo de tiempo.
- Habilita la cooperación, ya que todo el mundo tiene la misma imagen.
- Permite tomar decisiones de una manera más rápida y colaborativa.

Limitar el Trabajo en Progreso: al 100% de capacidad se tiene un rendimiento mínimo. Se asignan límites explícitos (WIP) de cuántos ítems puede haber en cada estado del flujo (columna). Esto evita la saturación y permite tener holgura para hacer modificaciones.
Se puede limitar la columna del backlog también. En este caso no se le llama WIP porque técnicamente todavía no empezó el trabajo, pero sirve para limitar el alcance a la capacidad real y no asumir compromisos imposibles.
Es una diferencia con SCRUM, donde se limita el tiempo del sprint a través del timebox.

Gestionar el Flujo de Trabajo: busca terminar el trabajo de la forma más fluida y predecible posible mientras se mantiene un ritmo sostenido. Limitar el WIP ayuda a predecir, y el seguimiento da información valiosa para las expectativas del cliente, predecir y mejorar.
El flujo está centrado en el cliente: es fin a fin, desde que entra al backlog hasta que se entrega.
El flujo es pull, no push. Eso quiere decir que un ítem que completa una fase no pasa a la siguiente automáticamente, sino que debe ser tomada y por un miembro del equipo que asume la responsabilidad de tratarla en el siguiente estado. Esto implica que deben haber columnas de acumulación por cada paso para permitir que los ítems liberen espacio de la fase de la que salieron pero no sean empujados a la siguiente fase sin que nadie las tome.

Hacer Políticas Explícitas: todas las políticas deben ser acordadas entre todas las partes involucradas, incluyendo a los clientes, interesados y trabajadores responsables del trabajo que está en el tablero. Las políticas deben estar expuestas en un lugar destacado.
Cada clase de servicio puede tener sus propias políticas
Las políticas deben ser:
- Pocas
- Sencillas
- Bien definidas
- Visibles
- Aplicables en todo momento
- Fácilmente modificables

Mejorar Colaborativamente:

- Iniciar con lo que hay ahora.
- Cambio colaborativo.
- Diseñar experimentos donde es seguro fallar y utilizarlos con el método científico para aprender.
- Mantener el cambio si el resultado es positivo.
- Volver fácilmente al estado anterior si el resultado es negativo.

Cómo Aplicar Kanban:

1. **Modelar el proceso:** identificar todas las etapas o pasos del proceso (las columnas).
2. **Definir la unidad de trabajo:** se divide el trabajo en piezas (como US). Se definen tipos de trabajo, que son qué es cada ítem y la capacidad asignada a cada uno. Ej: requerimientos 60%, mantenimiento 30%, defectos 10%.
3. **Definir el WIP**
4. **Definir políticas de calidad:** sobre:
    a. Reposición de trabajo en el tablero (cuándo, cuánta, por quién, etc.)
    b. Definir criterios de tracción (cuando una actividad está terminada)
    c. WIP
    d. Políticas para diferentes clases de servicio
    e. Cadencia de reuniones
    f. Otros principios y acuerdos
5. **Cuellos de botella y flujos:** identificar cuellos de botella y mover recursos para eliminarlos.
6. **Clases de servicio:** se definen grupos de ítems que pueden tener políticas en común y su propia definición de done. Ej: acelerada (urgente, puede saltarse el WIP), fecha fija, estándar e intangible (difícil de ver para el cliente, como mantenimiento):
7. **Cadencia:** de reuniones, releases, etc. Ej:
    - Team Kanban Meeting: diaria. Observar y seguir el estado y flujo del trabajo.
    - Team Retrospective: quincenal o mensual. Reflexionar sobre cómo el equipo gestiona el trabajo y cómo puede mejorar.
    - Team Replenishing Meeting: semanalmente o a demanda. Seleccionar elementos del backlog para realizar a continuación.

Métricas:

Métrica: indicador cuantificable que se toma para conocer el estado del desarrollo.
Las métricas no juzgan ni miden el desempeño de las personas. Sirven para tomar decisiones y balancear restricciones, renegociando con el cliente y estableciendo expectativas.
Las métricas deben ser simples, darnos información relevante que no teníamos y servir para mejorar. No son extrapolables entre proyectos, la utilidad depende del contexto y las métricas útiles en un proyecto pueden no serlo en otro.

Métricas en el Enfoque Tradicional: las métricas forman parte de una actividad de los procesos definidos que es el Monitoreo y Control. El enfoque tradicional no define qué

métricas tomar, se puede tomar una gran cantidad de métricas. Las métricas pueden ser cosas que se estiman o no. Una métrica puede ser la comparación entre la estimación y el valor real.
Se dividen en 3 dominios, que son el ámbito al que hacen referencia y son útiles:

- **Proceso:** consolidación de las métricas de varios proyectos despersonalizados y apuntando a la organización completa.
- **Proyecto:** permiten ver el estado actual del proyecto. Son útiles hasta que el proyecto termina. Todas las métricas de proyecto son después despersonalizadas para hacer métricas de proceso.
- **Producto:** permiten ver el estado del producto.

Métricas Básicas: son métricas que sirven de base para armar métricas concretas.

- **Tamaño:** métrica de producto que mide su tamaño. Encontrar una unidad de medida que refleje la realidad es complicado. Se puede usar cantidad de líneas de código, funcionalidades, requerimientos, etc.
- **Esfuerzo:** métrica de proyecto (y proceso) que mide cuántas horas lineales demanda el proyecto.
- **Tiempo Calendario:** métrica de proyecto y proceso que mide cuántos días dura cada tarea del proyecto.
- **Defectos:** métrica de producto que mide los defectos encontrados. Pueden estar clasificados.

Métricas en Enfoques Ágiles: se rigen por la filosofía de que las métricas son una salida, no una actividad en sí. Hay que medir sólo lo necesario. Se guían por los principios de satisfacer al cliente por medio de entregas contínuas y que la principal medida de éxito es el software funcionando.

Métricas en SCRUM: se definen únicamente 3 métricas a tomar:

- **Velocidad:** SP completadas y probadas por el PO por sprint. No se consideran US a medias. Se obtiene en el sprint review. En los primeros sprints la velocidad suele ser baja porque hay bastante incertidumbre y el equipo está familiarizándose con el proyecto, pero tiende a estabilizarse.
- **Capacidad:** cuánto trabajo puede completar el equipo en el próximo sprint. Se estima en el sprint planning. En los sprints avanzados se mide en SP en base a la velocidad, pero en los primeros sprints cuando no hay datos de velocidad o todavía no se estabilizó se usan las horas disponibles. En ese caso, se usa la descomposición por tareas del sprint backlog para estimar las horas. No se pueden relacionar horas con SP, porque horas solo mide esfuerzo y SP mide esfuerzo, complejidad e incertidumbre.
- **Running Tested Features (RTF):** es la cantidad de características que fueron probadas e implementadas. Se mide en el sprint review. No mide la complejidad, solo la cantidad, por lo que no mide el valor entregado. Se va acumulando sprint a sprint, y puede disminuir si cambia un requerimiento de algo ya implementado.

Métricas en Kanban:

- **Lead Time:** tiempo entre que un ítem particular entra al backlog hasta que se lo entrega. Es la vista del cliente, el tiempo que él ve desde que pide algo hasta que se le entrega. Es el ritmo de entrega. Se mide en días de trabajo.

- **Cycle Time:** tiempo entre el inicio y el final del proceso para un ítem en particular. Es la vista interna para el equipo. Es el ritmo de terminación. Se mide en días de trabajo.
- **Touch Time:** tiempo en el cual un ítem fue realmente trabajado. Cuánto tiempo estuvo en columnas de trabajo en curso.
- **Eficiencia del Ciclo de Proceso:** puede ser touch time sobre tiempo transcurrido o cycle time sobre tiempo transcurrido. Mide la eficiencia del proceso.

QA:

Calidad: todos los aspectos y características de un producto o servicio que se relacionan con su habilidad de alcanzar las necesidades manifiestas o implícitas. Es algo relativo a cada persona y contexto. Un software de calidad debe satisfacer:

- Las expectativas del cliente
- Las expectativas del usuario
- Las necesidades de la gerencia
- Las necesidades del equipo de desarrollo y mantenimiento
- Necesidades de otros interesados

Principios de la Calidad:

- La calidad está embebida, no se inyecta ni se compra luego de terminarlo.
- Es un esfuerzo colectivo
- Las personas son clave (necesidad de capacitación)
- Se necesita sponsor a nivel gerencial (recursos y apoyo)
- Se debe liderar con el ejemplo
- No se puede controlar lo que no se mide
- Simplicidad, empezar por lo básico
- La QA debe planificarse
- Aumentar los tests no aumenta la calidad
- Debe ser razonable para la organización

Perspectiva de la Calidad: la calidad debe estar desde todas las perspectivas de los interesados. Debe tener una visión trascendental que integre:

- Visión del usuario
- Visión del producto
- Visión basada en el valor
- Visión de manufactura

“Tipos” de Calidad:

- **Programada:** las expectativas y estándares que se establecen durante la planificación y el diseño. Son el objetivo.
- **Necesaria:** nivel mínimo de calidad para que el producto sea considerado correcto y satisfaga las expectativas del usuario.
- **Lograda:** es la calidad real que se alcanzó con el producto final.

Todo lo que no esté en la intersección de las 3 calidades es desperdicio o insatisfacción.

Calidad en Procesos: establecer y mantener estándares y procedimientos para llevar a cabo actividades de manera consistente y eficiente.

- Modelos para crearlos
- Modelos para mejorarlos
- Modelos para evaluarlos

Calidad en Productos: garantizar que los productos cumplan los estándares establecidos.

- Modelos para medirlos

Sobre el proyecto se aplican las técnicas de aseguramiento de calidad, pero terminan evaluando su adherencia al proceso o los artefactos que genera (el producto), por eso habla siempre de calidad de producto o proceso.

Aseguramiento de Calidad: asegurar que se alcancen los niveles requeridos de calidad para el producto de software. Implica la definición de estándares y procesos de calidad apropiados y asegurar que los mismos sean respetados.
Debería ayudar a desarrollar una "cultura de calidad" donde la calidad es vista como una responsabilidad de todos y cada uno.
Implica insertar en cada una de las actividades acciones tendientes a detectar lo más tempranamente posible oportunidades de mejora sobre el producto y sobre el proceso.

GAC: Grupo de Aseguramiento de la Calidad. Debe estar separado de la administración de proyectos para garantizar independencia.

- No debería reportar al gerente de proyectos
- No debería haber más de un nivel entre la gerencia de primer nivel y el GAC
- El GAC debería reportar a alguien realmente interesado en la calidad del software

Actividades de la Administración de Calidad:

- **Aseguramiento de Calidad:** establecer estándares y procedimientos. Decidir contra qué estándares se van a hacer las comparaciones (como auditorías y revisiones).
- **Planificación de Calidad:** selecciona los estándares y procedimientos a usar para un proyecto en particular y los modifica si es necesario (como el momento de las comparaciones). Define los productos de calidad deseados y el proceso de evaluación.
- **Control de Calidad:** ejecutar las actividades definidas en la planificación para asegurar que los estándares y procedimientos son respetados por el equipo. Dos enfoques:
    - Revisiones de calidad: método principal en el que un grupo revisa parte del proceso o producto para encontrar problemas.
        - Inspecciones para remoción de defectos (producto)
        - Revisiones para evaluación del progreso (ambos)
        - Revisiones de calidad (producto y estándares)
    - Evaluaciones de software automáticas y mediciones.

Funciones del Aseguramiento:

- Prácticas de aseguramiento: desarrollo de herramientas, métodos, estándares, etc. para las revisiones de calidad.
- Evaluación de la planificación del proyecto de software.

- Evaluación de requerimientos.
- Evaluación del proceso de diseño.
- Evaluación de las prácticas de programación.
- Evaluación del proceso de integración y prueba del software.
- Evaluación de los procesos de planificación y control de proyectos.
- Adaptación de los procedimientos de QA a cada proyecto.

Tipos de Estándares:
- De proceso: como debe ser implementado.
- De producto: características que los componentes deberían tener.

Calidad del Producto: no se puede sistematizar, no hay modelos generalizados para evaluar que se puedan aplicar como plantilla.
- **Aseguramiento:** revisiones técnicas y auditorías
- **Control:** testing

Modelos: son marcos conceptuales que se utilizan para evaluar y medir la calidad de los productos. Proporcionan un conjunto de características, criterios y métricas que se utilizan para determinar la calidad
- **Modelo de Barbacci/SEI:** la calidad se mide mediante 3 aspectos clave y busca el óptimo entre ellos:
    - Rendimiento
    - Seguridad
    - Confiabilidad
- **Modelo de McCall:** la calidad es un triángulo de 3 factores relacionados con las vistas de la calidad que se deben equilibrar:
    - Revisión del producto: capacidad de verificación, ej: facilidad mantenimiento, flexibilidad y prueba.
    - Transición del producto: facilidad con la que puede expandirse y crecer.
    - Operación del producto: calidad del producto en uso, ej: corrección, fiabilidad, usabilidad, eficiencia, etc.
- **Modelo ISO/IEC 25000:** 3 modelos:
    - Calidad en Uso: ISO 25010
    - Calidad de Producto: ISO 25010
    - Calidad de Datos: ISO 25012

Nunca se compara solo contra el modelo. Primero se define qué es lo que el cliente quiere como calidad del producto en requerimientos. La evaluación se centra más en cuanto se acerca a esos requerimientos que a los modelos teóricos.

Calidad del Proceso: el proceso es el único factor controlable para mejorar la calidad del software. Si se instancia un proyecto de un proceso que adopta la calidad, va a tener calidad embebida.
La calidad surge de la definición del proceso?

Procesos Basados en Calidad:
1. Se definen los procesos a seguir en el desarrollo del software para lograr la calidad.
2. Se aplican los procesos para desarrollar.

3. Se evalúa la calidad mirando artefactos y entregables.
4. Si no se alcanza la calidad esperada se revisa el proceso y se lo mejora.
5. Si se alcanza la calidad se estandariza el proceso, lo que implica documentar y comunicar los pasos, las actividades y las buenas prácticas utilizadas.

Modelos de Mejora: son recomendaciones o estándares para mejorar un proceso. Buscan elaborar un proyecto que tome un proceso y lo mejore. Se usan con los modelos de calidad para ayudar hacer el proyecto que mejore el proceso.

- SPICE (también es de calidad)
- IDEAL

IDEAL: es un proceso cíclico que busca la mejora contínua:

- **Initiating:** busca sponsoreo de la organización y estímulo.
- **Diagnosing:** determinar cuál es el punto de partida del proyecto. Hacer un análisis de brecha, documentar dónde está la organización y donde quiere ir.
- **Establishing:** se planifica y establece un plan de mejora detallado con objetivos, actividades necesarias y recursos asignados.
- **Acting:** se ejecuta el plan de acción en una prueba piloto.
- **Learning:** se documentan los resultados, se extrapola a toda la organización si fueron satisfactorios y se vuelve a empezar.

Modelos de Calidad ("modelos para crearlos"): referencia para bajar lineamientos que el proceso debe seguir para llegar a cierto objetivo. Entran al hacer un análisis de brecha. Son modelos de referencia para tener procesos de calidad.

CMMI: es una guía para mejorar la capacidad y madurez de los procesos organizacionales específicamente para organizaciones de software. Proporciona prácticas y criterios para ayudar a la organización a alcanzar niveles superiores de calidad, eficiencia y satisfacción del cliente.
Es un modelo flexible y adaptable que proporciona qué hacer, no cómo. No es una norma y no se certifica, se evalúa por profesionales reconocidos.
Tiene 3 constelaciones o ámbitos de mejora:

- **CMMI DEV:** guía para medir, monitorizar y administrar procesos de desarrollo.
- **CMMI SVC:** guía para entregar servicios internos o externos.
- **CMMI ACQ:** guía para seleccionar, adquirir y administrar productos y servicios.

Representaciones:

- **Por Etapas:** se definen 5 niveles de madurez según un conjunto de áreas de proceso con un conjunto de prácticas y capacidades que deben ser alcanzadas para pasar al siguiente nivel. Las organizaciones de nivel 1 son inmaduras y del nivel 2 para arriba maduras. Mide la madurez de la organización.
    - **Inicial (1):** la organización tiene procesos ad hoc no estandarizados. No existe visibilidad del proyecto. La organización actúa de forma reactiva a los problemas.
    - **Administrado (2):** la organización establece prácticas de gestión básicas. Inicia recopilación de métricas.

    - **Definido (3):** la organización establece y documenta procesos estandarizados y repetibles. Se definen roles y responsabilidades y existe retroalimentación.
    - **Cuantitativamente Gestionado (4):** establece mediciones cuantitativas para el control y la mejora de los procesos. Se establecen métricas y se recopilan datos para analizar el desempeño.
    - **Optimizado (5):** se enfoca en la mejora contínua y optimización de procesos.
- **Contínua:** tiene 6 niveles del 0 al 5 para cada área de proceso (un aspecto específico del desarrollo) que miden el grado de implementación de prácticas y metas para esa área. Mide la capacidad de cada área. Permite comparar distintas áreas y elegir el orden de las mejoras.

Grupos y Roles: los grupos se refieren a roles que cubren ciertas tareas. Los roles se adaptan al tamaño de la organización y al nivel de madurez deseado. Lo importante es que exista un responsable de cubrir las actividades de cada rol.

Áreas de Proceso por Nivel:

- **Administrado:**
    - Administración de requerimientos
    - Planeamiento de proyectos
    - Monitoreo y control de proyectos
    - Administración de acuerdos con el proveedor
    - Mediciones y análisis
    - Aseguramiento de calidad del proceso y el producto
    - Administración de configuración
- **Definido:**
    - Desarrollo de requerimientos
    - Solución técnica
    - Integración de producto
    - Verificación y validación
    - Foco en el proceso organizacional
    - Definición del proceso organizacional
    - Capacitación organizacional
    - Administración integrada de proyectos
    - Administración de riesgos
    - Análisis y resolución de decisiones
- **Cuantitativamente Administrado:**
    - Performance del proceso organizacional
    - Gestión cuantitativa del proyecto
- **Optimizado:**
    - Administración del performance organizacional
    - Análisis causal y resolución

Diferencias entre Ágil y CMMI:

- **Filosofía y Enfoque:**
    - **Ágil:** se basa en principios ágiles y la flexibilidad, respuesta al cambio y entrega de valor incremental.

    - **CMMI:** basado en un enfoque más tradicional con prácticas estandarizadas y con planificación y control exhaustivos. La calidad del producto sale de la calidad del proceso.
- **Gestión de Proyectos:**
    - **Ágil:** iteraciones cortas y frecuentes. Se enfoca en la colaboración con el cliente, retroalimentación y adaptabilidad. La planificación es flexible.
    - **CMMI:** procesos planificados y controlados con énfasis en la definición y el seguimiento de requisitos, la planificación detallada, el monitoreo del progreso y el control de cambios.
- **Entrega de Software:**
    - **Ágil:** entrega temprana y contínua de software funcional y de calidad para adaptarlo a las necesidades del cliente.
    - **CMMI:** se enfoca en la entrega final con los aspectos de calidad, cumplimiento de requisitos y procesos establecidos.
- **Flexibilidad y Adaptabilidad:**
    - **Ágil:** adaptabilidad y flexibilidad para responder a los cambios en los requisitos y las necesidades del cliente.
    - **CMMI:** marco estructurado y detallado de prácticas y procesos que deben seguirse.

Técnicas de Verificación y Validación:
- Revisiones Técnicas
- Pruebas de Software

Revisiones Técnicas: proceso de verificación y validación estático que busca detectar defectos y corregirlos en las etapas tempranas del desarrollo. Se aplican en varios momentos del desarrollo y puede inspeccionarse cualquier representación legible del software. Hecha por colegas.
Ventajas:
- Pueden descubrirse muchos errores
- Pueden inspeccionarse versiones incompletas
- Pueden considerarse otros atributos de calidad

Desventajas:
- Es difícil introducir inspecciones formales
- Sobrecargan al inicio los costos y solo ahorran cuando el equipo gana experiencia
- Requieren tiempo para organizarse y frenan el desarrollo

Costos:
- **Infraestructura:**
    - Entrenamiento
    - Desarrollo de plantillas e informes
    - Desarrollo de guías de lectura
    - Implantación de programas de medición
    - Herramientas de soporte
- **Operacionales:**
    - Tiempo individual y grupal
    - Tiempo en completar informes
- **Adicionales:**

    - Preparar material, arreglar calendario, recolectar datos, etc.
    - Tiempo dedicado a la mejora de calidad

Tipos de Revisiones:

- **Inspecciones:** buscan detectar y remover todos los defectos eficiente y efectivamente. Tienen un proceso formal, checklists, mediciones y fase de verificación.
- **Walkthrough:** busca tener una sobrecarga mínima, la capacitación de desarrolladores y el retorno rápido. Es un proceso informal con poca o ninguna preparación y sin mediciones.

Métricas de ejemplo:

- Cantidad de defectos
- Densidad de defectos
- Esfuerzo de la inspección
- Esfuerzo por defecto
- Porcentaje de inspecciones
- Porcentaje de defectos corregidos

Inspección: es una actividad de QA que busca:

- Descubrir errores
- Validar que el software alcanza sus requisitos
- Garantizar que el software sigue los estándares establecidos
- Conseguir software desarrollado de manera uniforme
- Hacer que los proyectos sean más manejables

Se llevan a cabo mediante una reunión y el éxito depende de su planificación.
Son la forma más barata y efectiva de encontrar fallas. Proveen métricas al proyecto y mejoran la calidad del producto.
No son formas de encontrar soluciones a las fallas o formas de evaluar la aprobación del producto o el desempeño de las personas.

Roles:

- **Autor:**
    - Creador o encargado de mantener el producto
    - Inicia el proceso asignando el moderador y junto a él el resto de los roles
    - Entrega el producto al moderador
    - Informa tiempo de retrabajo y número de defectos al moderador
- **Moderador:**
    - Planifica y lidera la revisión
    - Selecciona el resto de los roles con el autor
    - Entrega el producto a los inspectores con tiempo antes de la reunión
    - Coordina la reunión
    - Hace el seguimiento de los defectos detectados
- **Lector:** lee el producto
- **Anotador:** registra los hallazgos de la inspección
- **Inspector:** examina el producto antes de la reunión y registra su tiempo de preparación

Proceso:

- **Planificación:** elegir equipo, preparar material y calendario.
- **Visión General:** presentar proceso y producto.
- **Preparación:** análisis individual para encontrar posibles defectos.
- **Reunión de Inspección:** análisis del equipo para encontrar posibles defectos previos y filtrar falsos positivos. Se hace un registro e informe de revisión con qué se revisó, por quienes, qué se descubrió y conclusiones. La decisión puede ser:
    - Aceptar el producto sin modificaciones
    - Aceptar el producto provisoriamente
    - Rechazar el producto
- **Corrección:** corregir defectos.
- **Seguimiento:** verificar correcciones y recolectar datos.

Duración: máximo 2 horas.

- Planificación: 15-30 minutos.
- Vista previa: 500 líneas por hora.
- Preparación: 100-140 líneas por hora.
- Inspección: 125-140 líneas por hora.
- Mejora del proceso: 30 minutos.
- Tamaño máximo de inspección: 250-280 líneas.

Valores para código y documentación respectivamente.

Recorrida/Walkthrough: técnica de análisis estático en la que un diseñador o programador dirige miembros del equipo de desarrollo y otros interesados a través del producto, y los participantes formulan preguntas y realizan comentarios acerca de posibles errores, violación de estándares de desarrollo y otros problemas.

Comparación de Revisiones:

- **Pruebas:**
    - Caras de mejorar
    - No sirven para remover errores en etapas tempranas
- **Inspecciones:**
    - Mejores resultados, procesos controlados
    - Métricas útiles a lo largo de todo el ciclo de vida del desarrollo
- **Walkthroughs:**
    - Buenos resultados pero pocas métricas
    - No hay control del proceso

Auditorías de Software: evaluación independiente (no depende del mismo departamento, externo al equipo) de los productos o procesos de software para asegurar el cumplimiento de estándares, lineamientos, especificaciones y procedimientos basada en un criterio objetivo que especifique:

- La forma o contenido de los productos a ser desarrollados
- El proceso por el cual los productos son desarrollados
- Cómo debería medirse el cumplimiento con estándares y lineamientos

Beneficios:

- Evaluar el cumplimiento del proceso de desarrollo
- Determinar la implementación efectiva de:

    - El proceso de desarrollo organizacional
    - El proceso de desarrollo del proyecto
    - Las actividades de soporte
- Dar visibilidad a la gerencia sobre los procesos de trabajo

Tipos de Auditoría:
- Proyecto
- Configuración Funcional
- Configuración Física

Auditorías de Proyecto: validan en cumplimiento del proceso de desarrollo. Verificar objetivamente la consistencia del producto a lo largo de su evolución en el proceso de desarrollo. Se llevan a cabo de acuerdo a lo establecido en el plan de aseguramiento de calidad del software. Las inspecciones de software y revisiones de documentación deberían incluirse en estas auditorías.
Determinan que:
- Las interfaces de software y hardware sean consistentes con los requerimientos de diseño en la ERS.
- Los requerimientos funcionales de la ERS se validan en el plan de verificación y validación de software.
- El diseño del producto satisface los requerimientos funcionales a medida que evoluciona.
- El código es consistente con el diseño detallado del software.

Auditorías de Configuración Funcional: compara el software construído con los requerimientos. Busca validar que implementa sólo y completamente esos requerimientos. Se valida también que la matriz de rastreabilidad esté actualizada.

Auditorías de Configuración Física: compara el código con la documentación de soporte. Asegura que la documentación que se entrega es consistente con el código desarrollado. Verifica cada ítem de configuración con su documentación y solo se entrega el software cuando se corrijan las desviaciones.

Roles:
- **Gerente de QA:**
    - Prepara el plan de auditorías
    - Calcula el costo
    - Asigna los recursos
    - Responsable de resolver las disconformidades
- **Auditor:**
    - Acuerda la fecha de la auditoría
    - Comunica el alcance
    - Recolecta y analiza la evidencia objetiva que es relevante y suficiente para tomar decisiones del proyecto
    - Realiza la auditoría
    - Prepara el reporte
    - Realiza el seguimiento de los planes de acción
- **Auditado:**

- Acuerda la fecha de la auditoría
- Participa de la auditoría
- Proporciona evidencia al auditor
- Contesta al reporte
- Propone el plan de acción para deficiencias encontradas
- Comunica su cumplimiento

Proceso de Auditoría:

- **Preparación y Planificación:**
  - Solicitar auditoría (auditado)
  - Acordar lugar y fecha (ambos)
  - Acordar alcance y objetivo (ambos)
  - Proponer asistentes (auditado)
  - Acordar participantes (ambos)
  - Solicitar documentación preliminar (opcional) (auditor)
  - Revisar documentación preliminar (auditor)
  - Preparar documentación e información requerida para la auditoría (auditado)
- **Ejecución:**
  - **Reunión de Apertura:**
    - Presentar a los participantes (auditor)
    - Revisar cronograma, alcance y objetivo de la auditoría (ambos)
    - Informar el proceso (auditor)
    - Informar el carácter de la auditoría y formalismos (auditor)
    - Plantear inquietudes (auditado)
    - Clarifica dudas sobre el proceso (auditor)
  - **Ejecución propiamente dicha:**
    - Realizar preguntas (auditor)
    - Contestar preguntas (auditado)
    - Solicitar evidencia (auditor)
    - Mostrar evidencia (auditado)
    - Completar el checklist (auditor)
    - Informar fecha de envío del reporte (auditor)
- **Análisis y Reporte del Resultado:**
  - **Evaluación de los resultados (auditor):**
    - Revisar el checklist
    - Revisar las anotaciones
    - Revisar la documentación solicitada
    - Realizar una verificación cruzada de las desviaciones contra el proceso
    - Realizar el informe preliminar
  - **Reunión de cierre:**
    - Presentar reporte preliminar (auditor)
    - Solicitar clarificación de resultados (auditado)
    - Explicar resultados (auditor)
    - Acordar resultados (ambos)
    - Definir fecha de envío de reporte final
  - **Entrega del reporte final (auditor):**
    - Actualizar reporte preliminar

            - Preparar el reporte final
            - Guardar una copia
            - Enviar el resultado al auditado
- **Seguimiento:**
    - Analizar desviaciones (auditado)
    - Preparar plan de acción (auditado)
    - Enviar plan de acción indicando fecha de realización (auditado)
    - Revisar plan de acción (auditor)
    - Aprobar plan de acción (auditor)
    - Realizar seguimiento de las acciones (auditado)
    - Informar cumplimiento (auditado)
    - Verificar cumplimiento (auditor)
    - Cerrar las desviaciones (auditor)
    - Informar el cierre de las desviaciones (auditor)

Checklist de Auditoría:

- Fecha
- Lista de auditados con su rol
- Nombre del auditor
- Nombre del proyecto
- Fase actual del proyecto (si aplica)
- Objetivo y alcance de la auditoría
- Lista de preguntas

Reporte de Auditoría:

- Identificación de la auditoría
- Fecha de la auditoría
- Auditor
- Auditados
- Nombre del proyecto
- Fase actual del proyecto
- Lista de resultados
- Comentarios
- Solicitud de planes de acción

Tipos de Resultados:

- **Buenas Prácticas:** práctica, procedimiento o instrucción que se desarrolló mejor de lo esperado. Se usan cuando el auditado:
    - Estableció un sistema efectivo
    - Desarrolló un alto grado de conocimiento y cooperación interna
    - Adoptó una práctica superior a cualquier otra vista
- **Desviaciones:** requieren un plan de acción por el auditado. Son:
    - Cualquier desviación que resulta en la disconformidad del producto con algún requerimiento
    - Falta de control para satisfacer los requerimientos
    - Cualquier desviación del proceso definido o los requerimientos que cause incertidumbre sobre la calidad del producto

- **Observaciones:** condiciones que deberían mejorarse pero no requieren plan de acción. Son:
    - Opiniones acerca de una condición incumplida
    - Prácticas que deben mejorarse
    - Condiciones que pueden resultar en futuras desviaciones

Métricas de Auditoría:
- Esfuerzo por auditoría
- Cantidad de desviaciones
- Duración de la auditoría
- Cantidad de desviaciones clasificadas por área de proceso de CMMI

Testing: proceso destructivo de tratar de encontrar defectos, que se presume existen, en el código. Se debe asumir que hay algo incorrecto, ir con una actitud negativa. El testing tiene éxito cuando se encuentran defectos.
El testing colabora con el control de calidad de producto y proceso, pero NO garantiza calidad, porque no previene los defectos, los detecta. La calidad debería estar embebida en el software desde el inicio.

Error: algo incorrecto que se detecta en la misma fase en la que se introduce.
Defecto: error no detectado que se detecta en una fase posterior.
Falla: mal funcionamiento del software. No todas las fallas son culpa de defectos y no todos los defectos causan fallas.

Severidad de un Defecto: que tanto afecta al sistema. Cada organización o equipo decide los niveles. Testing asigna la severidad. Ejemplo:
- **Bloqueante:** no permite ejecutar ninguna funcionalidad.
- **Crítico:** una funcionalidad no anda.
- **Mayor:** hay al menos un camino alternativo que anda.
- **Menor:** falla una parte poco importante u opcional.
- **Cosmético:** no afecta al funcionamiento.

Prioridad: que tan urgente es resolverlo para el cliente. El cliente asigna la prioridad. Ejemplo:
- Urgente
- Alta
- Media
- Baja

La severidad es distinta a la prioridad y no es proporcional. Ejemplo de defecto cosmético con prioridad urgente: sistema contable que no muestra los montos grandes con puntos separadores de millares.

Niveles de Pruebas: cuánto se abarca en las pruebas.
- **Tests Unitarios:** Primer nivel de testing. Se prueban aspectos puntuales de un componente individual de forma aislada tras su construcción. Se hace con acceso al código y con apoyo del entorno de desarrollo como frameworks, y es automatizable.

Suele ser hecho por los desarrolladores en el workflow de implementación. Se suelen encontrar errores y se reparan sin reportes de incidentes.

- **Tests de Integración:** Segundo nivel de testing. Prueba que componentes que funcionan bien de forma aislada lo hagan en conjunto. Se hace una integración de manera incremental. Teóricamente deberían ser parte del workflow de prueba, pero suelen ser hechos en la implementación por un senior dev o tech lead. Es el primer paso del CI/CD. La salida es un Build.
- **Tests de Sistema:** Tercer nivel de testing. Prueban que el sistema completo (Build) funcione (recuperación de fallas, seguridad, rendimiento, etc.). Investigan requerimientos funcionales y no funcionales. El entorno de prueba debe parecerse lo más posible al de producción. Según Meles en clase: se usa en procesos en cascada, no aplica a procesos iterativos.
- **Tests de Aceptación:** Cuarto nivel de testing. Prueba del usuario para determinar si la aplicación se ajusta a sus necesidades. No está orientada a encontrar defectos. Abarcan pruebas en entornos de prueba y en producción.

Los niveles 2, 3 y 4 son verticales porque afectan varias capas del sistema. El nivel 2 también es horizontal porque afecta a varios componentes de la misma capa.

Ambiente: conjunto de herramientas de software, desde el OS en el hardware, necesarias para que el sistema funcione.

- **Desarrollo:** entorno local donde los desarrolladores construyen el sistema. Tiene herramientas de desarrollo y es flexible. Es donde ocurren las pruebas unitarias.
- **Prueba:** ambiente limpio independiente del de desarrollo para evitar el problema de “anda en mi máquina”. Simula en parte al entorno de producción. Es donde se hacen las pruebas de integración.
- **Pre-Producción:** simula el entorno de producción para las pruebas finales, como las pruebas de sistema y de aceptación.
- **Producción:** es el ambiente donde el software está funcionando y es accesible a los usuarios finales. También se hacen pruebas de aceptación beta en este ambiente.

Caso de Prueba: conjunto de variables y condiciones para determinar si el sw está funcionando. Dice qué acciones debo realizar, qué datos debo ingresar y qué condiciones se tienen que cumplir para obtener un resultado determinado.
Los casos de prueba permiten reproducir defectos.

Los casos de prueba deberían elaborarse en base a documentación del cliente o a los requerimientos, y en última instancia en base al código. Los casos de prueba solo verifican, no validan.
No se pueden hacer casos de prueba infinitos, así que hay que probar la mayor cantidad de escenarios con la menor cantidad de casos de prueba.

Condición de Prueba: reacción esperada del sistema frente a un estímulo constituído por varias entradas. Las condiciones de prueba se enuncian antes que los casos, y cada condición debe estar probada por al menos un caso de prueba.

Estructura de un Caso de Prueba:

- **Objetivo:** la característica del sistema a probar.

- **Datos de entrada y ambiente:** datos a introducir al sistema que se encuentra en condiciones preestablecidas.
- **Comportamiento esperado:** la salida o acción esperada del sistema de acuerdo con los requerimientos. Puede ser un fallo.

Estrategias:
- Caja Negra
- Caja Blanca

Caja Negra: se definen las entradas y salidas de los casos de prueba basadas en el comportamiento esperado del sistema sin ver como funciona por dentro. Es una estrategia dinámica.
Se dividen en dos grupos:
- Basados en especificaciones:
    - Partición de equivalencias
    - Análisis de valores límites
- Basados en la experiencia:
    - Adivinanza de defectos
    - Testing exploratorio

Testing de Caja Negra Basados en Especificaciones: ambas estrategias se basan en probar las clases de equivalencia. Usan la misma definición, pero cambian cómo armar los casos de prueba.

Clase de Equivalencia: conjunto de valores de una condición externa (de entrada o salida) que hacen que la funcionalidad a probar se comporte igual (misma condición de prueba). La granularidad depende de la cantidad de casos de prueba que se quiera tener.

Partición de Equivalencias: se hace un caso de prueba por cada combinación de clases de equivalencia. Se toma cualquier valor de la clase para la prueba. Hay un caso por combinación.

Análisis de los Valores Límite: se eligen valores en los límites de las clases de prueba (el último valor permitido y el primero no permitido). Se puede hacer también una prueba con un valor central o asumir que si los extremos funcionan el centro también. Se hace un caso de prueba por cada límite de cada clase.

Caja Blanca: se analiza la estructura del código para elaborar los casos de prueba. Permiten garantizar la cobertura de tests. Pueden detectar partes ineficientes, caminos inútiles, partes maliciosas, etc. Es estático.

Ciclo de Pruebas: ejecución de todos los casos de prueba aplicados a una versión del sistema.

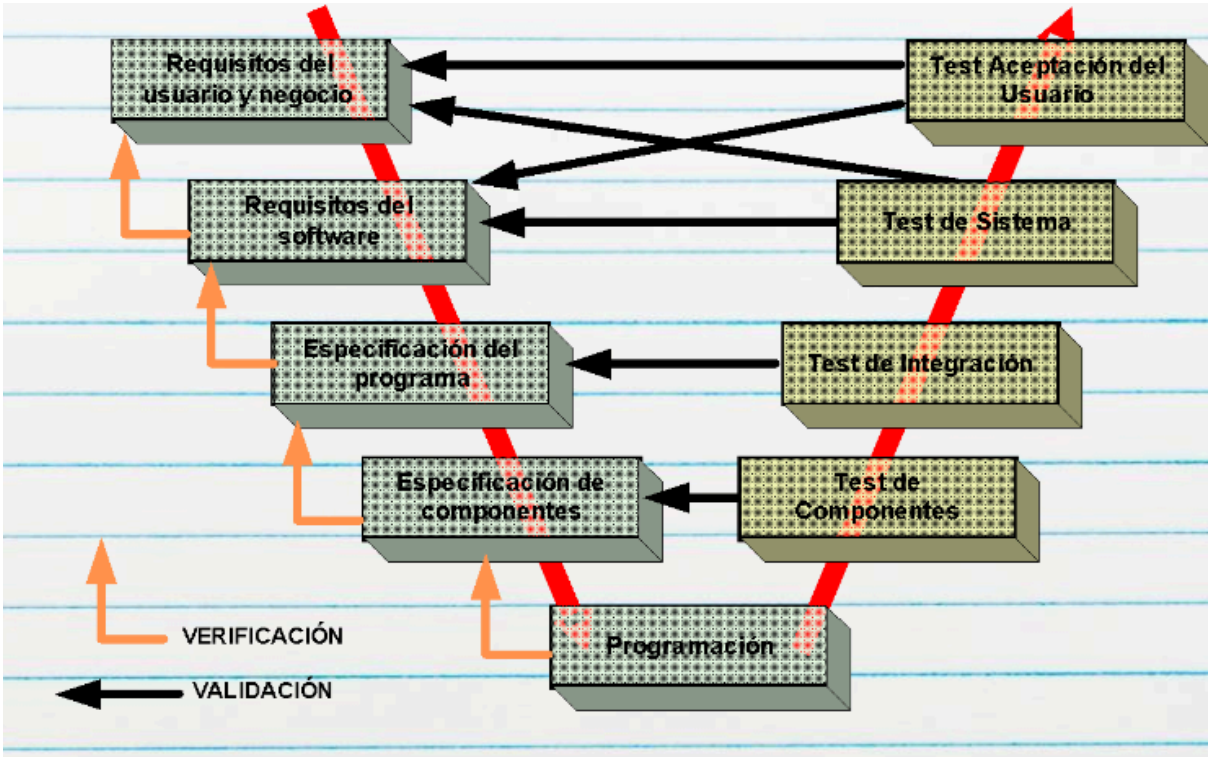
Regresión: al concluir un ciclo de prueba y corregir los defectos se verifica el sistema completo otra vez, no solo las partes que fallaban.

Proceso de Pruebas Tradicional:

- **Planificación:** verificar que se entienden los objetivos del cliente y stakeholders, y elaborar el plan de testing. Se deciden los criterios y se toman decisiones.
- **Identificación:** diseñar y priorizar los casos de prueba y el ambiente de prueba.
- **Especificación:** lo mismo.
- **Ejecución:** ejecutar el ciclo de pruebas, lo que incluye desarrollar los casos, crear los datos de prueba, automatizar, ejecutar y registrar.
- **Análisis de Fallas:** evaluar los criterios de aceptación y los resultados obtenidos. Si las pruebas fallan se vuelve a ejecución. Si pasan se termina.
- **Fin**

Objetivos de Testear de Manera Temprana:
- Dar visibilidad temprano al equipo de cómo se va a testear el producto.
- Disminuir el costo de corregir defectos.

Modelo en V:



Cuánto Testing es suficiente: depende del riesgo y los costos del proyecto.
Cuándo una fase de Testing fue completada: se usa un criterio de aceptación. Ej:
- Costos
- Porcentaje de tests que pasan
- No tener defectos de cierta severidad

Principios del Testing:
- El testing muestra defectos.
- El testing exhaustivo es imposible.
- Es mejor el testing temprano.

- Paradoja del pesticida: los defectos pueden quedar ocultos a los mismos casos de prueba que se ejecutan siempre.
- El testing es dependiente del contexto.
- Es una falacia pensar que no hay errores.
- Una unidad de programación no debería probar sus desarrollos.
- El testing NO empieza al terminar la implementación.
- El testing NO es probar que el sistema funcione.
- El testing NO garantiza calidad del producto ni del proceso.

Tipos de Pruebas:

- **Smoke Test:** primera corrida de los tests que verifican que el sistema no produce una falla catastrófica.
- **Regresión:** probar el sistema luego de corregir un defecto para verificar que la corrección no rompió otra parte que previamente funcionaba.
- **Prueba de Sanidad:** prueba rápida que una corrección puntual eliminó los defectos y no introdujo errores nuevos.
- **Prueba Funcional:** las pruebas se basan en funciones y características y su interoperabilidad con sistemas específicos.
    - Basadas en requerimientos
    - Basadas en procesos de negocio
- **Prueba No Funcional:** prueba como funciona el sistema.
    - Prueba de rendimiento
    - Prueba de estrés
    - Prueba de carga
    - Prueba de usabilidad
    - Prueba de mantenimiento
    - Prueba de fiabilidad
    - Prueba de portabilidad

Testing Ágil: no es algo que se ejecuta al final, es parte y está embebido en el proceso de desarrollo y es una actividad que aporta valor contínuo, influencia el desarrollo y guía el diseño. A diferencia del testing tradicional sí busca prevenir defectos, no encontrarlos (romper el sistema).

Manifiesto Ágil del Testing: 5 valores, 9 principios y 6 prácticas concretas.

Valores:

- Pruebas durante sobre pruebas al final
- Prevenir defectos sobre encontrar defectos
- Entender lo que se está probando sobre verificar funcionalidad
- Construir un mejor sistema sobre romper un sistema
- Responsabilidad de todo el equipo con la calidad sobre responsabilidad de un tester

Principios:

- El testing se mueve hacia adelante en el proyecto
- El testing no es una fase
- Todos hacen testing
- Reducir la latencia del feedback

- Las pruebas representan expectativas
- Mantener el código limpio y corregir los defectos rápido
- Reducir la sobrecarga de documentación de las pruebas
- Las pruebas son parte del done
- Desarrollo conducido por pruebas

Prácticas Concretas:
- Pruebas unitarias y de integración automatizadas
- Pruebas de regresión a nivel de sistema automatizadas
- Pruebas de exploración
- TDD
- ATDD (aceptación)
- Control de versión de las pruebas con el código

Rol del Tester: no es un rol estático, tiene alta incidencia en el equipo y se vincula con todos los roles. Vinculaciones:
- PO: le ayuda en
    - Preparar y revisar historias
    - Definir criterios de aceptación
    - Entender mejor el dominio del negocio
    - Identificar dependencias
- Equipo de sistemas:
    - Prepara documentación de historias a probar
    - Difunde plan, casos y datos de prueba
    - Ayuda a conocer la completitud de la funcionalidad
    - Recibe bugs y comunica al PO y devs
- Devs:
    - Define estrategias de prueba
    - Automatiza pruebas funcionales
    - Detecta y notifica bugs
    - Feedback temprano de las historias
    - Sugiere mejoras funcionales y de usabilidad
- Tester (él mismo):
    - Preparar y mantener ambiente de pruebas
    - Hacer casos y datos de pruebas de las historias
    - Validación del done de las historias
    - Registra y notifica bugs
    - Aceptación de la historia con el PO
    - Hacer métricas de calidad

Diferencias entre Test tradicional y Ágil:
- El testing se hace durante el desarrollo y lo guía, no al final.
- El testing lo hace todo el equipo, no un tester solo. Tampoco hay auditorías (los equipos son autoorganizados, las auditorías son externas).
- El testing ágil busca prevenir defectos, el testing tradicional romper el sistema.
- La automatización es un pilar para lograr CI.
- La pirámide de testing está invertida.

Cuadrantes del Testing Ágil: tipos de testing con 4 dimensiones de calidad a tener en cuenta.

<table>
<tr><td>Automatizadas y manuales</td><td colspan="2">Orientadas al negocio</td><td>Manuales</td></tr>
<tr><td rowspan="2">Guían el desarrollo</td><td>Tests funcionales
UX
Prototipos
Simulación</td><td>Tests Orientados al Usuario
Exploración
Flujos de trabajo
Usabilidad
Aceptación
Monitoreo</td><td rowspan="2">Critican al producto</td></tr>
<tr><td>Tests Unitarios y de Componentes</td><td>Tests de Calidad y Requerimientos No Funcionales</td></tr>
<tr><td>Automatizadas</td><td colspan="2">Orientadas a la tecnología</td><td>Automatizadas con herramientas</td></tr>
</table>

Pirámide de Testing Ágil:

- El 80% son pruebas unitarias de los componentes
- El 15% son pruebas de aceptación y funcionales en la capa de servicio y las APIs
- El 5% son pruebas end to end a través de la interfaz gráfica

En el tradicional está invertido y sale test de integración en vez de aceptación.

Beneficios de la Automatización:

- Ejecución rápida
- Reusabilidad
- Mayor cobertura
- Mayor precisión
- Mayor alcance
- Reducción de costos

TDD: técnica de desarrollo en donde primero se diseña, luego se escriben los tests y al final se desarrolla la funcionalidad. Involucra dos prácticas:

- Test First Development: pensar como voy a probar algo antes de desarrollarlo.
- Refactoring: modificar código ya creado para ajustarlo a algún criterio.

Leyes del TDD:

- No escribir nada de código hasta no tener las pruebas.
- No escribir más tests de los necesarios para que falle.
- No escribir más código del necesario para que pasen los tests.

Pasos del TDD (Ciclo Red-Green-Refactor):

- Escribir un test

- Hacer que el test compile
- Hacer que falle (Red)
- Hacer el código para que pase
- Correr el test y que pase (Green)
- Refactorizar para limpiar el código (no cambia funcionalidad, emprolija)

Patrones:

- **Red Bar:**
  - **One Step Test:** escribir un solo test que verifique una pieza mínima de funcionalidad.
  - **Starter Test:** primer test que te ayuda a explorar la API o el comportamiento deseado.
  - **Explanation Test:** escribir comentarios para aclarar qué se espera del código.
  - **Learning Test:** permite conocer APIs externas de las que no sabemos su comportamiento.
  - **Another Test:** añade un nuevo test para cubrir un caso adicional o una variante.
  - **Regression Test:** agrega un test que reproduce un bug descubierto para evitar regresiones futuras.
- **Green Bar:**
  - **Fake It:** devolver un valor fijo o solución falsa para pasar el test.
  - **Triangularización:** introducir un segundo test con diferentes datos para forzar una implementación más general.
  - **Obvious Implementation:** cuando la solución es trivial implementarla directamente, sin fake it.
  - **One to Many:** resolver tests para colecciones de elementos.